

Business Capability

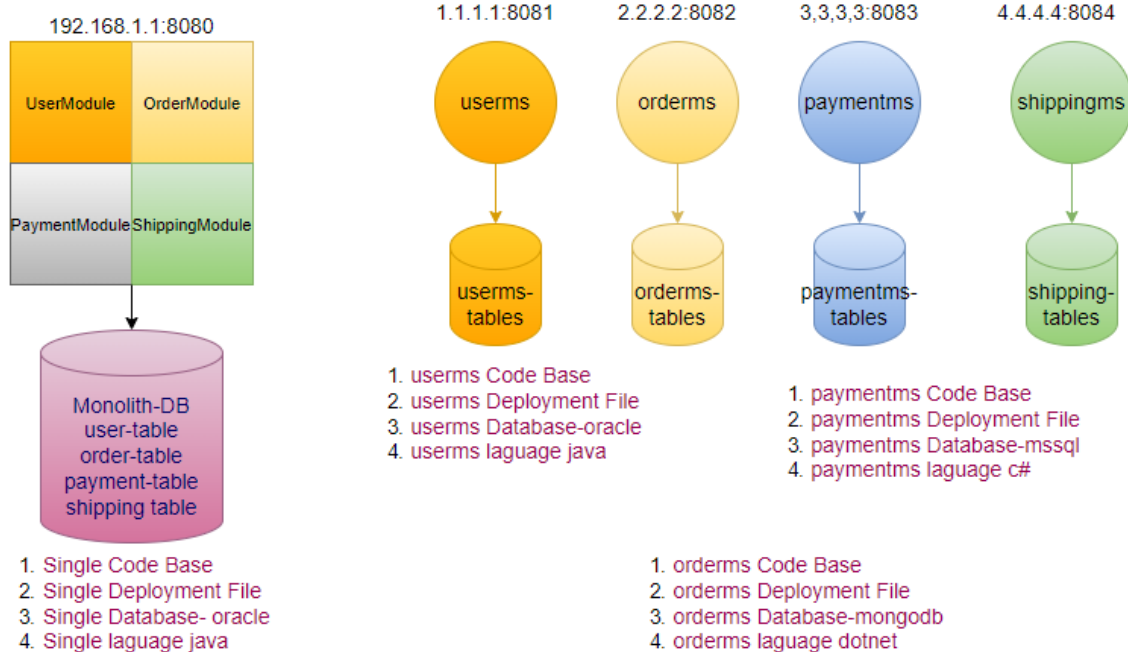
- The microservices approach to division is different. Services are split up and organized around business capability.
- Such services take a broad-stack implementation of software for that business area which includes:
 - User-interface
 - Persistent storage, and
 - Any external collaborations.

Decentralization:

- Monolithic applications prefer a single logical database.
- Enterprises often prefer a single database across a range of applications.
- Microservices prefer letting each service manage its own database.
- With data distribution comes challenges of distributed transactions.
- As each service lives in its own space, transaction boundaries span across multiple services and requires a compensating transactions in case of failures.

Any microservice can talk to any microservices (REST, messaging), there is no centralization in ms. They can talk to each other without any interpreter (service bus).

Decentralization at DB level:



UC: something happens with userdb, failure of one DB will not impact the other flows.

Every MS allows to call only its own DB, suppose if usersms wants to know about the orders then usersms call Rest Service of ordersms to get the order details, can't call order db directly. Every MS is owner of its own DB.

Advantage

- Scalable
- Higher Throughput
- Resilient
- DB Heterogeneity

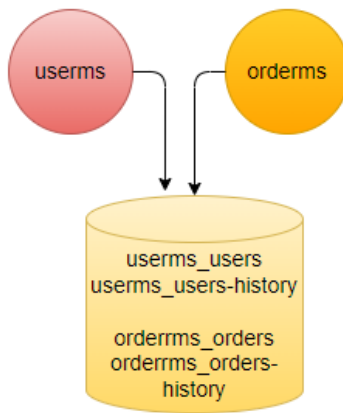
Disadvantage

- **Cost:** Increase
- **Maintenance:** Increase
- **Constraints:** No Relationship between DB

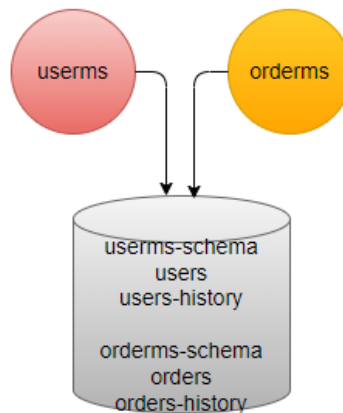
Database Design Strategy

Database Design Strategy

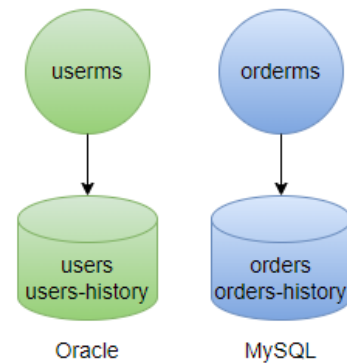
1) table Segregation



2) Schema Segregation



3) Database per service



1) Table Segregation (Most economical)

Table: `ms-name_table`

No Relationship between different `ms-name_table`

2) Schema Segregation

schemas are independent to each-other

3) Database Segregation

Every ms has their own DB.

No relationship between any DB

How to relate data (correlation maintain) between two different db?

Identifier may be present in other DB.

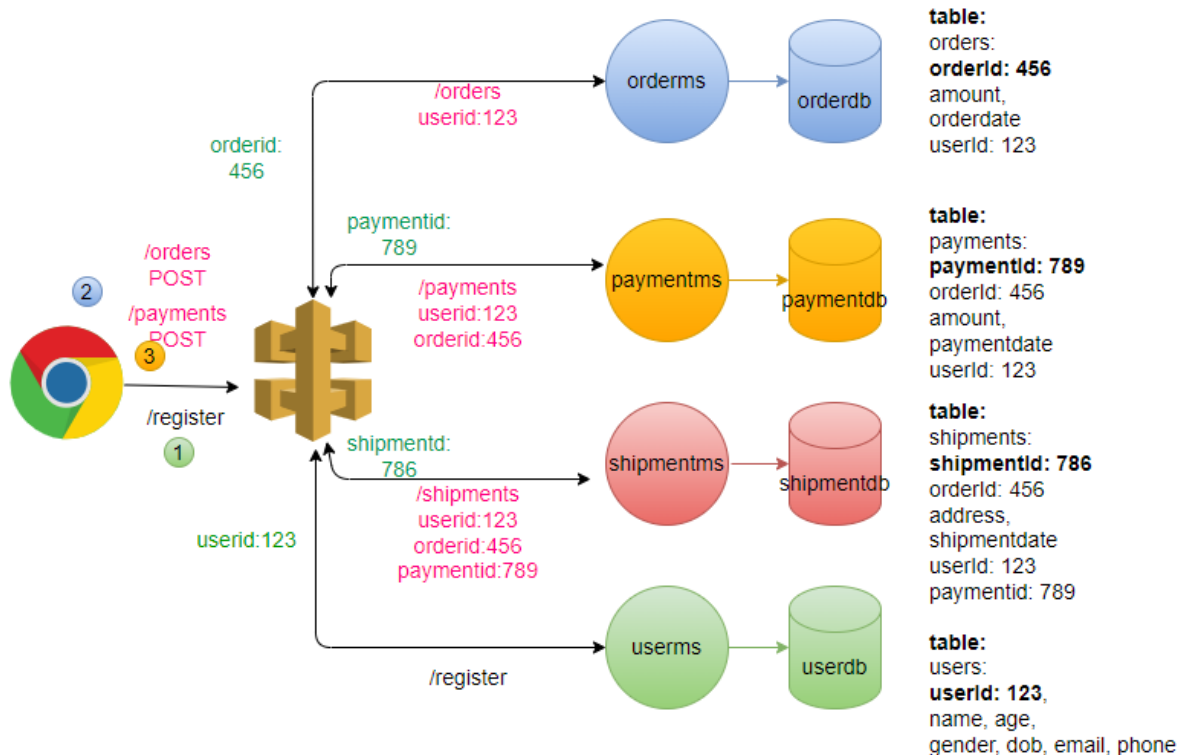
DRY: Don't repeat yourself

WET: Write Everything twice

step1: user registered in the system (userid will be generated)

step2: registered user calls the ordersms, will send all details with the request in header (will send the userid).

step3: registered user calls the payments for his order, sends all details with the request in header (will send the userid, orderid).



Rule:

- paymentms can't be called unless order is created.
- ordersms can't be called unless user is onboarded
- Identifier taken care by ms itself.

How to Aggregation of Data happen?

Aggregation done at MS level not at DB, Aggregator as a separate service.

Transaction Management:

Local Transaction:

In Single DB (Atomicity-either all or none)- Transaction (executing group of Database operations as a one single logical operation) ACID

Distributed Transaction:

2-phase commit

1. Prepare Phase: Transaction mgr will ask the participant nodes whether they are ready to commit their individual local transactions
2. Commit Phase: If all the participating nodes reply positively then transaction manager will tell the nodes that commit your transactions

Limitation: 2PC (Transaction Manager)

Transaction Coordinator has to wait for confirmation from all the nodes

SAGA: Distributed transaction for microservices using different DB

- Choreography
- Orchestrator

Choreography:

- Every service participates in its own local transaction. If any service transaction fails, responses back to service received the event, so that those services will generate a compensatory transaction that roll back

Orchestrator:

- There will be an orchestrator that manages entire transaction among the services.
- Orchestrator call services sequentially, if it receives success response from service, it calls another service. If one of the services fail then orchestrator will rollback all previous service transactions.
- Camunda/Apache Camel (mini ESB)

OLTP: Transaction (real-time)- quick + relatively small set of data

level. In case of OLTP, report service pulls the data from other services, not suitable for large set of data. Implement at active DB level.

OLAP Analytics(batch/stream)

batch: All data already there (historical data)

stream: Unbound (data stream) we don't know how much data keep flowing and we process: Spark, Storm, Flink, Kinesis

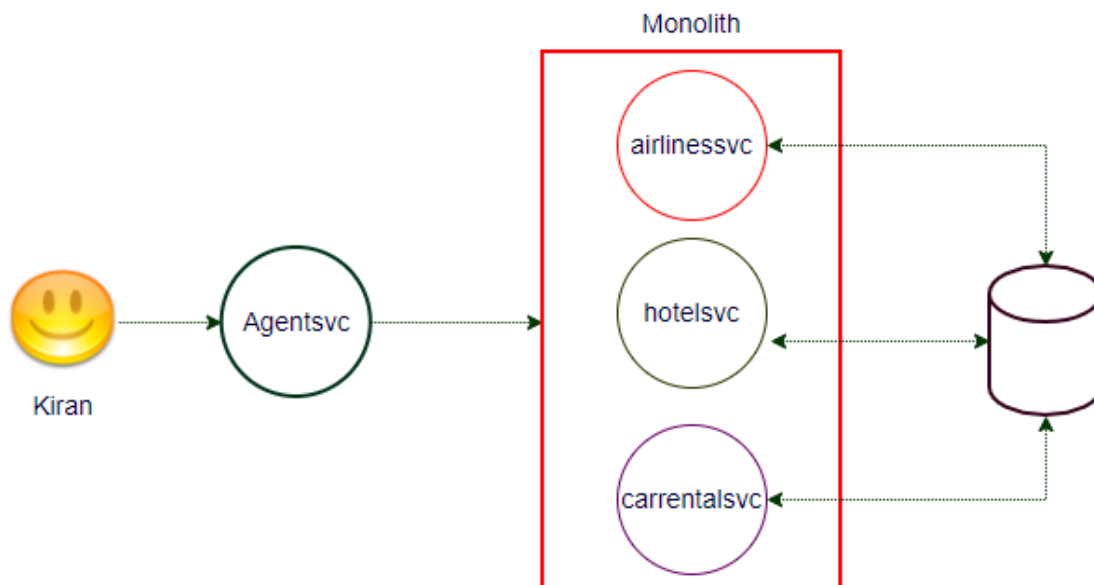
In case of OLAP, data kept in other data warehouse (Redshift)

periodically data pushed into data warehouse through some asynchronous process.

Transaction Management

Monolith: Two phase commit (same db).

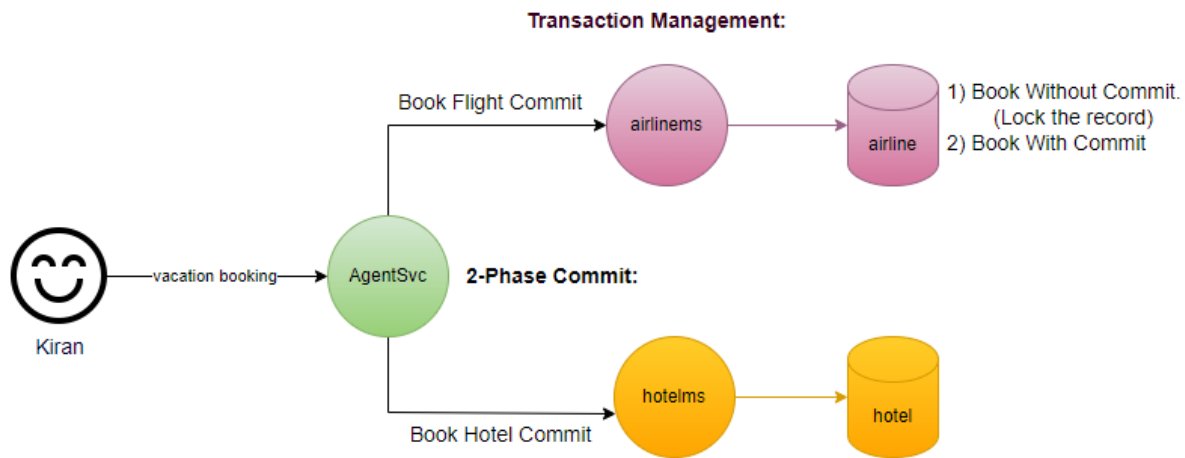
UC: vacation booking system



UC1: booking successfully committed.

UC2: carrentalsvc failed, rollback the hotelsvc and airlinesvc.

Microservices: Each service has their own database using the commander.



Rollback in monolith was easy (with in the same DB), but in microservices is not very difficult, in second phase commander issues rollback command rather than the commit command, it keeps the data in **inconsistent state**.

Transaction Management:

Transaction Management managed in microservices using the SAGA pattern, using the **compensatory action**.

SAGA: Orchestrator

SAGA: Choreograph

SAGA: (using messaging): Finite State Machine

We need one more component orchestration is called **SAGA Execution coordinator**.

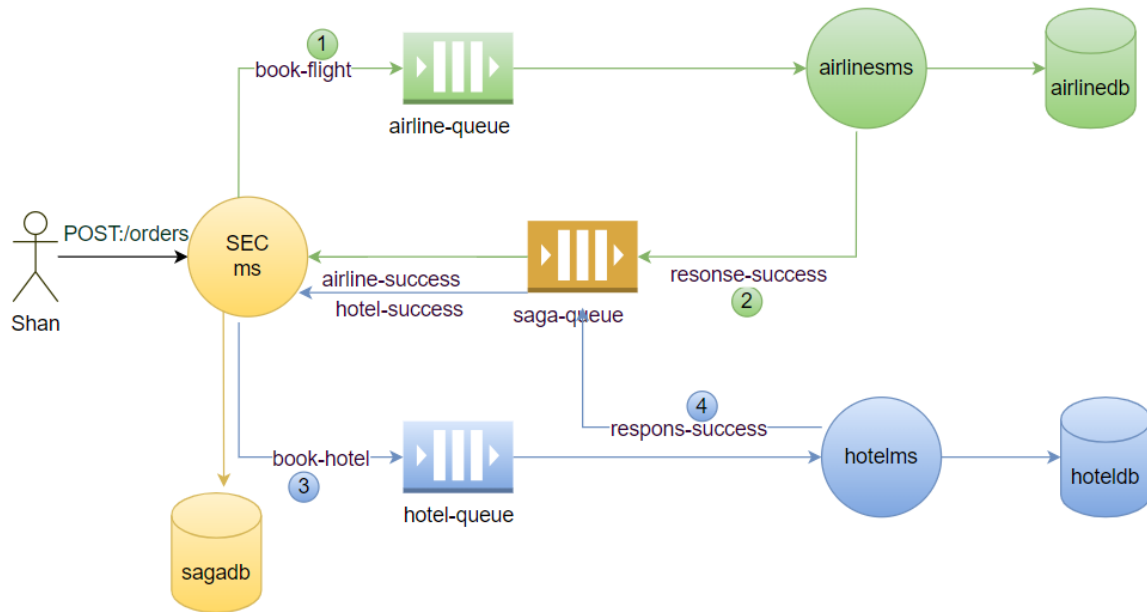
SAGA Patterns: Eventual Consistent: Not support 2-phase commit or rollback.

For every transaction there are always two outcomes (success/failure)

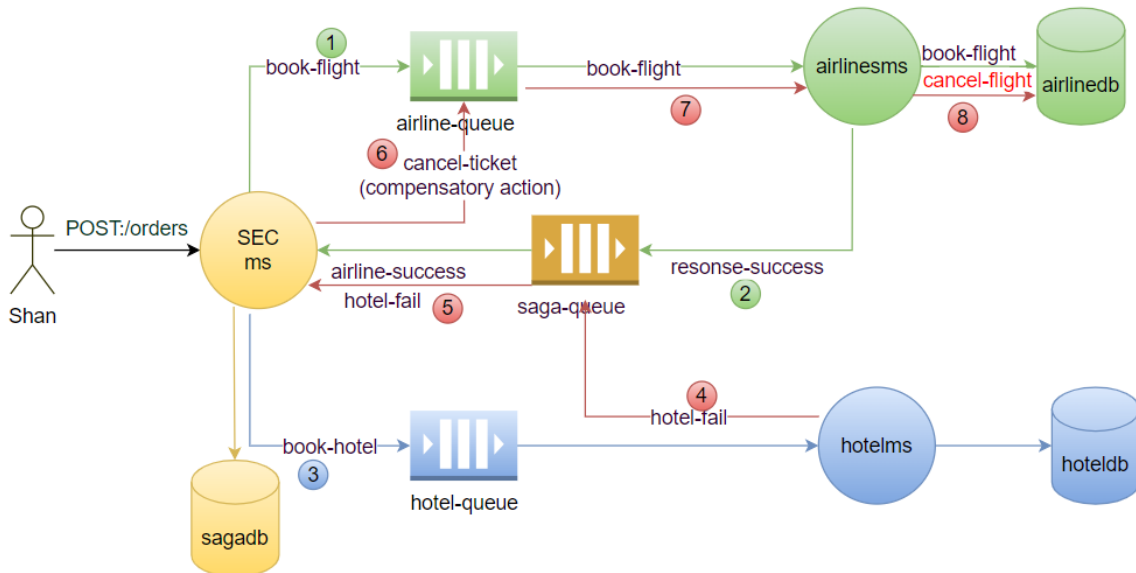
SAGA (Transaction Management)

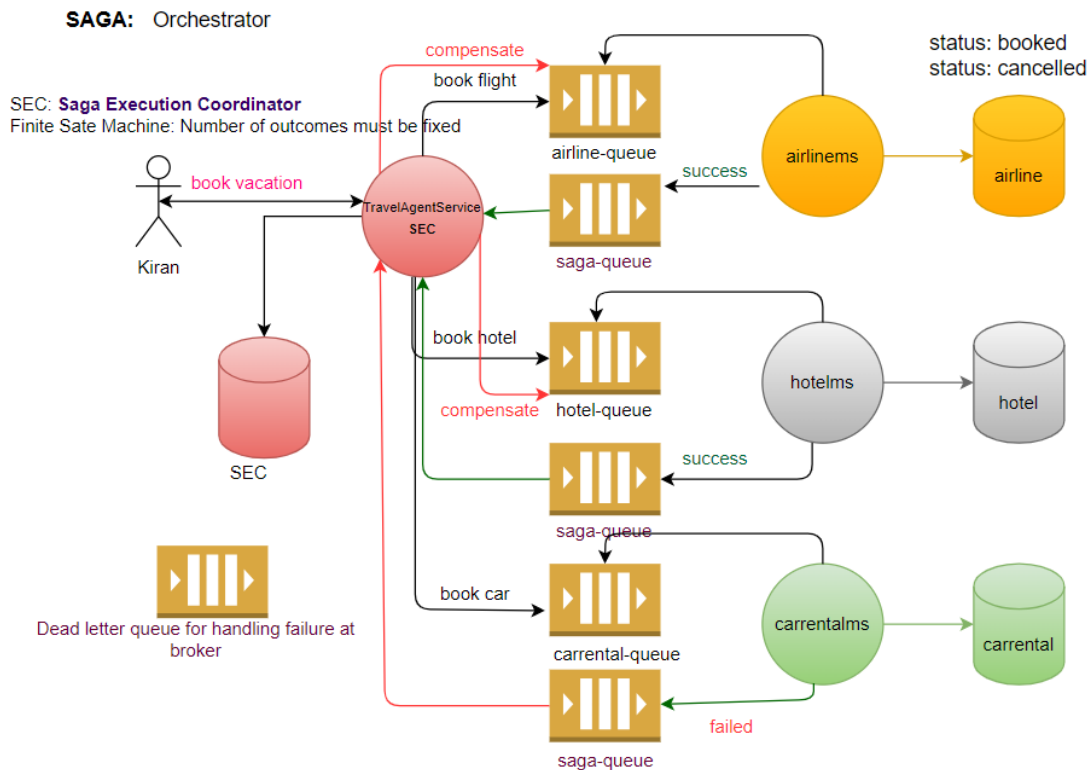
SEC: Saga Execution Coordinator(Eventual Consistent)-Orchestrator
All the workflow event/state logic

UC: All Success



UC: Hotel Failed





Two outcomes:

1. airline-success → hotel-booking
2. airline-failed ⇒ TripBooking Failed

1. hotel-success ⇒ carrental-booking
2. hotel-failed ⇒ compensate-airline -> TripBooking Failed

1. carrental-success ⇒ TripBooking Success
2. carrental-failed ⇒ Compensate Hotel -> compensate-airline -> TripBooking Failed

It's an Event driven pattern, not an Event sourcing. Event sourcing comes with CQRS.

It's all about handling the transaction, not from customer point of view when customer cancel the ticket, in that case charges are deducted will be another svc.

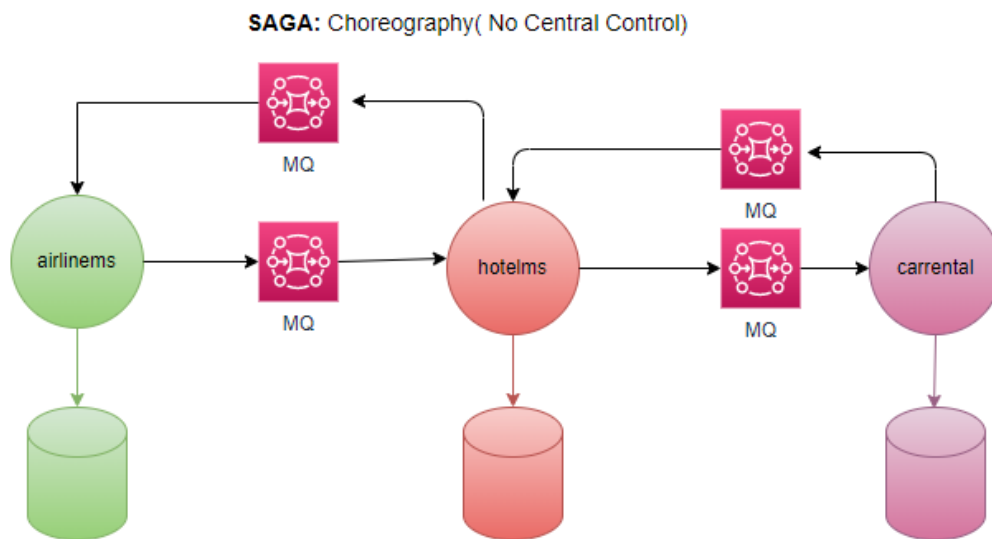
step1: Travel Agent Service (SEC) receives a request from Kiran for new transaction (vacation plan).

step2: Travel Agent Service first send the request to airline-queue, airlines receives the request, if ticket available then update the airline-db and responses back to saga-queue, it will send the response back to SEC.

step3: After receiving response from airlinesvc Agent Travel Service sends the request to hotel-queue, hotels receives the request, if room available then update the hotel-db and responses back to saga-queue, it will send the response back to SEC. Each transaction is local (completed), there is no rollback.

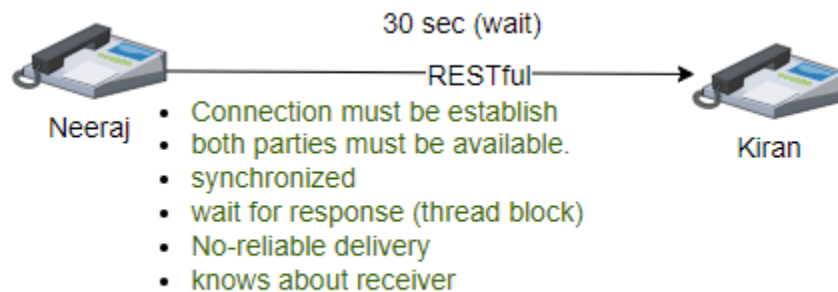
In case of failure response from hotels, SEC sends the compensatory-action request to airlines-queue, airlines receives the request and update the status of ticket “booked” to the status ticket “canceled” and responses back to saga-queue. If any transaction is failed in that case, SEC issues a compensatory-action for each of the previous services in the chain.

SAGA(Choreography)

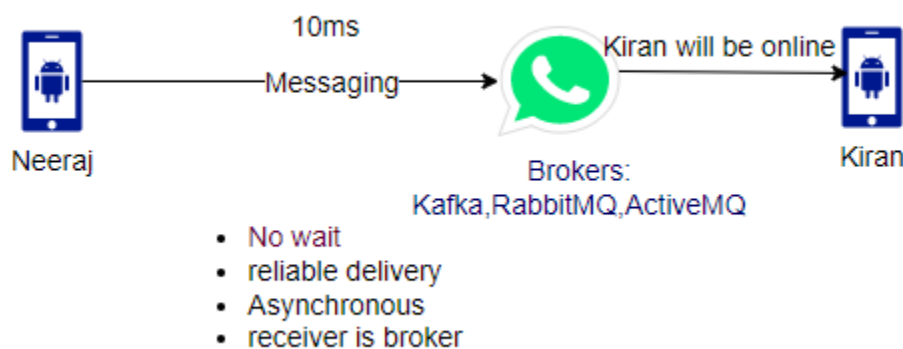


Inter-process communication Ways of communications?

Synchronous (RESTful)



Asynchronous (Messaging- guaranteed to be delivered)



Messaging Solution

ActiveMQ- Push based (Broker has to do push the message): AMQP

RabbitMQ – Push based, good for medium traffic: AMQP

Apache/Confluent Kafka- Pull based (consumer pulls the message), good for heavy traffic, high scalable, high throughput and low latency: Kafka

AWS SQS- Push based

AWS SNS- Pull based

AMQP: Advanced Messaging Queue Protocol (ActiveMQ, RabbitMQ)- PUSH

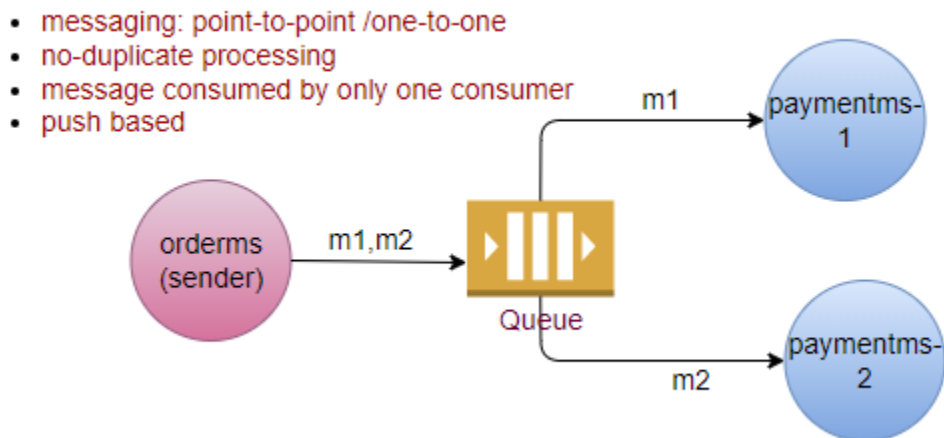
Kafka: Own protocol (Kafka Protocol)- PULL

JMS

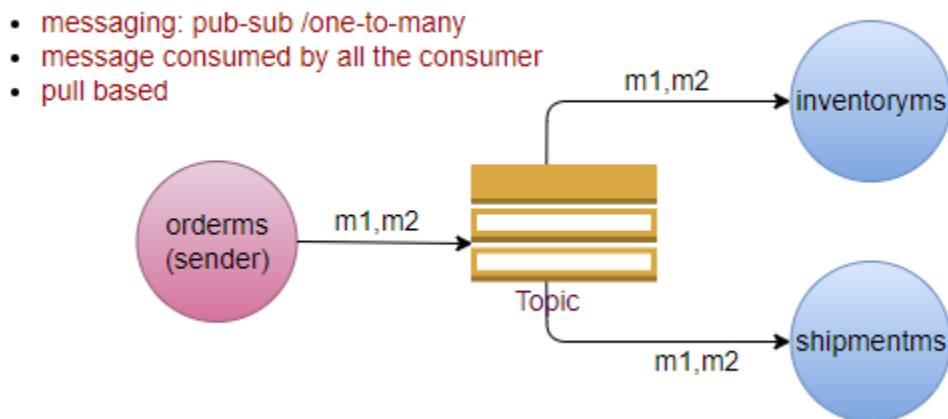
Messaging Types:

Point-to-Point(one-to-one) Queue: Only one receiver (**push based**)- SQS

UC: orders published the message for paymentms, if there are multiple instances of paymentms, we want only should be received by only 1 instance, don't want duplicate processing.



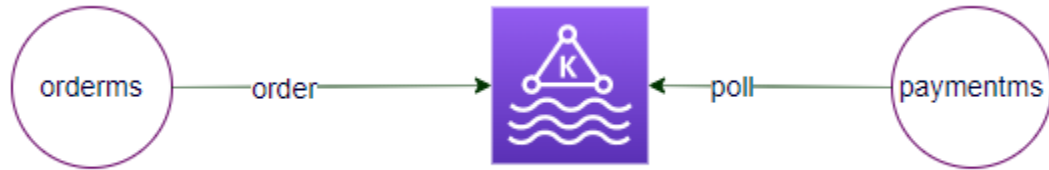
Pub-Sub (one-to-many) Topic (deliver only if consumer available): Multiple receiver (**pull based consumer** poll the message)- SNS



Pub-Sub Kafka:

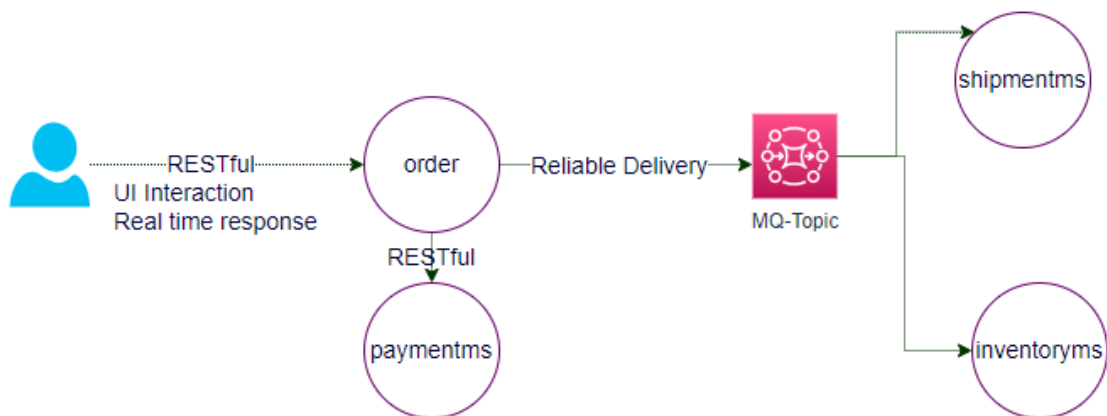
- Poll message
- High throughput

- Low Latency
- High Scalability
- Keeps message for 7 days (reliable)



When go for RESTful and when go for Messaging?

It depends upon the use case to use case



High Availability: close to zero downtime **Immediate consistency**

UC: multiple instances of users (in case of one instance is down other instance available to response the request), but from database point of view its more complex.

In to make database HA, create a one more database instance (**All the writes on DB must be gone synchronously**), means once users writing on primary db, it also writes data on secondary database also. It slightly impacts the performance.

Why not Asynchronous writes?

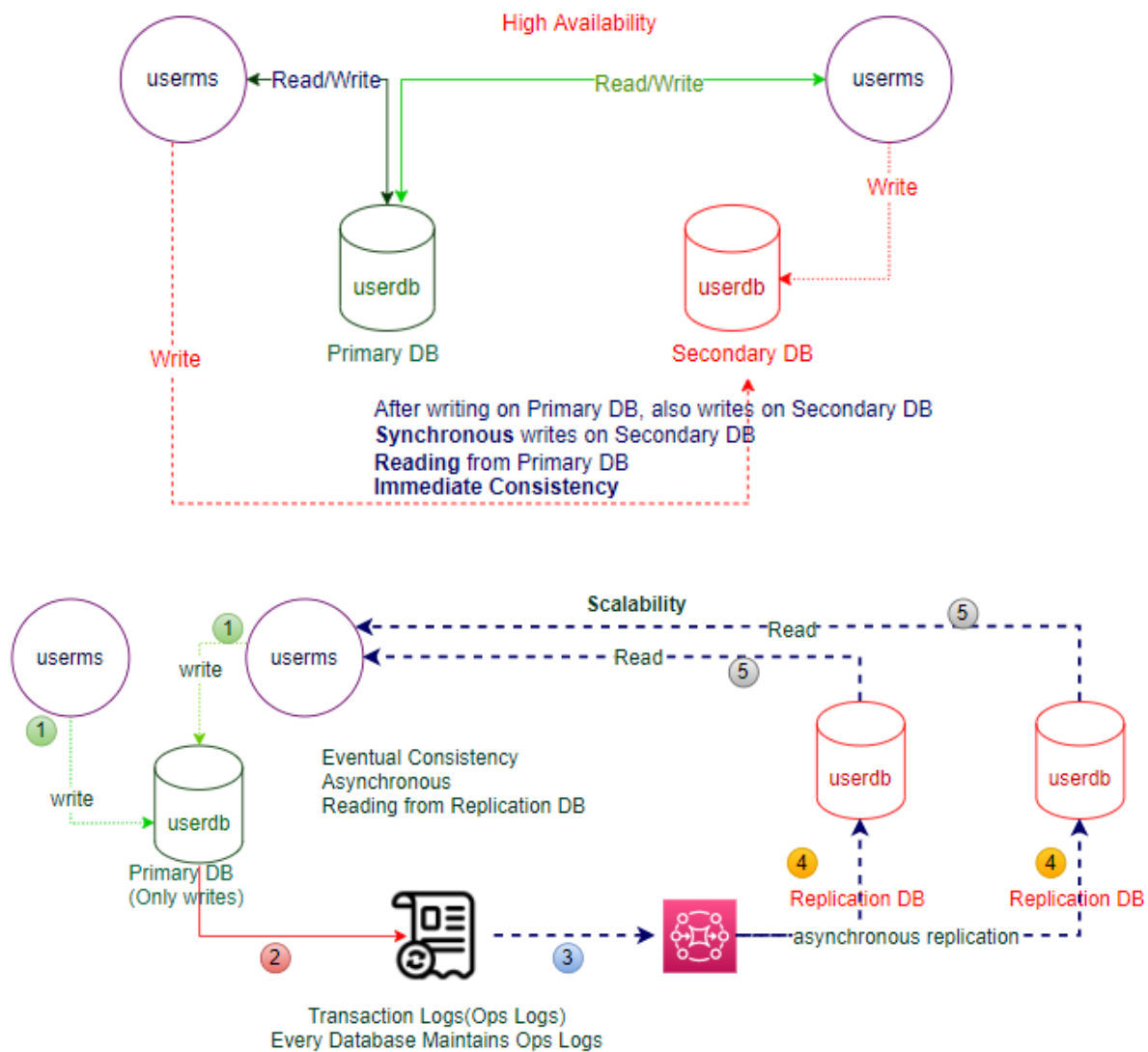
In case of primary DB down, users immediately start reading from secondary database. There will be switch from primary DB to secondary DB in case of

primary DB down. Both DB must have same data, we can't miss even a single transaction.

Always reading from primary DB.

How to improve the performance? **Eventual consistency (HA)**

Scalability: Always reading from replica DB.



High Availability and Scalable Database (AWS RDS)

Service Provided by Cloud (AWS)

